

Advanced Programming

Python

15.06.2026

Marius Kiskemper

Marius.Kiskemper@atos.ai

Atos

GUIs & Interfaces

01

Interfaces

What is an Interface?

Definition

- Interface is the interaction point of a computer program with the user
 - Or even the interaction point between two computer programs
- Interface is defined by the needed parameters, return type, ...
- Most common types of interfaces: CLI (Command Line Interface), GUI (Graphical User Interface)
 - Both are the interaction point for a user to execute a program
- Every time when you call / execute something (method, class, whole script) you use an interface

Interface in a class vs. a whole program

Difference

From Lecture 3: class & method interfaces

- These are interaction points for a computer program
 - the interface of a class / a method is what you need to provide to interact with it
 - `ex1 = ExampleClass(attr1, attr2, attr3)` → provide three different attributes
 - `returned_string = example_method(param1, param2, param3)` → provide three different parameters

Now: interfaces for a whole program

- CLI interface: `python carRental.py --rent Car2, Customer4, 15 --return Car2, Customer4, Vendor1, False, 92`
- GUI interface: having a graphical interface where you click buttons and fill input boxes to provide the parameters needed to execute the program

→ Interfaces are **any point in a program where an interaction** (class call, method call, script execution, ...) **could occur**

→ A GUI is a special type of interface

02

GUI Basics

What is a GUI?

Definition

- GUI stands for Graphical User Interface
- The connection between a user and the computer
- Display information, optical interaction points
 - → opposite to text-based interfaces (cli-call or python script executions)
- Should be very intuitive to use
- Giving immediate feedback
- Should be user-centric designed
- Takes longer to develop than other interfaces

Why build a GUI?

Benefits

- Makes code execution accessible for everyone (non-technical users, users with no domain expertise, ...)
- Makes user interactions easy and fast to explain → users knows exactly what each part is used for
 - Icons (trash bin, check-button, ...)
 - Drag-and-drop fields
 - Text fields
- Most user friendly type of interface
- Visualizing results
- → Making code solutions accessible for everyone
- Nearly every computer program today is theoretically spoken a GUI
- → Is needed when the program will be accessed by non-technical users, but as an interface for other programmers is mostly unnecessary

Elements of a GUI

Examples

- Buttons
- Input Box
- Icon
- Images
- Menu
- Menu bar
- Popup-Window
- All organized in windows

03

UI vs. UX

Style vs. functionality

Different GUI-building approaches

UI

User Interface

- Optimize the visual appearance of the GUI → creating the visually most appealing GUI
- Colour schemes
- Font selection
- Object themes
- Transfer messages (e.g. brand identity) via the appearance
- Create interactive elements
- Present all contents in an aesthetically pleasing way

UX

User experience

- Maximize the interaction quality for the user → creating the best possible experience
- Simple workflows
- Easy understandable
- Intuitive layout
- Avoid unnecessary steps
- Incorporate what the user needs in the most effective way
- Seamless and meaningful user interaction with the GUI

04

Implementation in Streamlit

Implementation in Streamlit

General

- Streamlit turns a plain Python script into a web app → „Web Development“ without HTML, CSS, JavaScript, ...
- Aimed at Data Scientists, Engineers and researchers who want an interactive UI fast → not front-end developers.
- Trade-off: extremely productive for dashboards, demos, prototypes, internal tools
 - NOT meant for complex multi-page consumer products
- Run any app with:
 - `streamlit run app.py`
- An app is just a Python file. There is no "GUI builder" — the layout is the order of the code.
- Keep usual modular function & responsibility design
- Keep data & logic in own modules
- Use data type checking (with error raising) & feedback for good UX

Implementation in Streamlit

General

- Every interaction (button click, slider move, text entry) **reruns the entire script from top to bottom**
- There are no event callbacks (like in Tkinter)
 - Not: "on click do X"
 - Instead: write a linear script that describes the page *for the current state*
- Consequence: **plain variables reset on every run**
 - For stateful variables: use `st.session_state`

```
# This counter does NOT work – resets to 0 on every rerun:
count = 0
if st.button("Add"):
    count += 1
st.write(count)    # always shows 0 or 1, never accumulates

# Correct: persist across reruns with session_state
if "count" not in st.session_state:
    st.session_state.count = 0
if st.button("Add"):
    st.session_state.count += 1
st.write(st.session_state.count)    # accumulates correctly
```

Implementation in Streamlit

Session State

- The only way to persist data across reruns.
- Behaves like a dictionary and allows attribute access:

```
st.session_state["key"] = 1    # dict style
st.session_state.key = 1       # attribute style
```
- Initialise once with the "guard" pattern:

```
if "fleet" not in st.session_state:
    st.session_state.fleet = build_fleet()
```
- Widgets can write into session_state via a key:

```
st.text_input("Name", key="username")
st.write(st.session_state.username)
```
- Force a refresh after changing state programmatically:

```
st.rerun()
```
- Use session state for heavy workloads (only do them once)

Implementation in Streamlit

Outputs & Inputs

Outputs / Displaying

```
st.title("My App")
st.header("Section"); st.subheader("Subsection")
st.write("Anything: text, numbers, DataFrames, charts")
st.markdown("**bold**, *italics*, `code`")
st.code("print('hi')", language="python")
st.metric("Speed", "200 km/h", delta="+10")
st.dataframe(df); st.table(df)
st.error("..."); st.warning("..."); st.success("..."); st.info("...")
```

Inputs

```
name = st.text_input("Name")
age = st.number_input("Age", min_value=0, value=18, step=1)
agree = st.checkbox("I agree")
many = st.multiselect("Pick several", ["X", "Y", "Z"])
level = st.slider("Level", 0, 100, 50)
clicked = st.button("Submit")
file = st.file_uploader("Upload")
```


Implementation in Streamlit

Layouting

```
# Sidebar – great for navigation / global controls
with st.sidebar:
    page = st.selectbox("Page", ["Home", "Settings"])

# Columns – side-by-side content
col1, col2, col3 = st.columns(3)
col1.metric("A", 1)
col2.metric("B", 2)
with col3:
    st.button("Go")

# Tabs – group related views
tab1, tab2 = st.tabs(["Chart", "Data"])
with tab1:
    st.line_chart(df)

# Expander – hide detail until needed
with st.expander("Show advanced"):
    st.write("...")

# Container & placeholders
placeholder = st.empty()
placeholder.write("Loading...")
# later:
placeholder.write("Done!")
```

07

Exercise Time

Exercise Time

Vehicle Rental

- Build a modular, efficient and user friendly GUI (using Streamlit) for your implemented vehicle class
 - Purpose: Display the characteristics of the individual vehicles and be able to interact with their attributes
 - If inheritance not finished yet, use the separate vehicle classes:
 - [WDSKI25A-Advanced-Programming/Exercise_3 at main · Marius2311/WDSKI25A-Advanced-Programming](#)
- If you also implemented the class Person with subclasses, you can start building a GUI for the rental process

Meme of the day



Next Lecture



- 1. Coding Guidelines**
- 2. Object Oriented Programming**
- 3. Inheritance**
- 4. GUIs & Interface**
- 5. Code optimization**